

VOLE-ACT: Post-Quantum Anonymous Credit Tokens from VOLE-in-the-Head and MAYO

Samuel Schlesinger

July 2026

Research draft — definitions and analysis are stated as targets;
no complete security reduction is claimed

Abstract

Anonymous credit tokens let an issuer grant a client a hidden, spendable balance: the client can later redeem credits unlinkably, each redemption consumes a one-time nullifier and returns a fresh token on the remaining balance. Classical constructions rest on discrete-logarithm-style algebra — blind or randomizable signatures, algebraic MACs, Sigma protocols — none of which survives a quantum adversary. We present VOLE-ACT, a post-quantum candidate instantiation of the Katz–Schlesinger anonymous-credit design [24] whose entire symmetric layer is Keccak and whose only algebraic assumption concerns the multivariate-quadratic (whipped MAYO) trapdoor. The central idea is to *avoid blindness entirely*: after accepting a revealed SHAKE256 commitment digest, the issuer salts and signs a wrapper target; possession of the signature is later proven in zero knowledge inside a VOLE-in-the-Head proof system that is native to exactly the two ingredients that must be proven — Keccak (a Boolean circuit) and MAYO verification (a quadratic system over $\mathbb{F}_{16} \subseteq \mathbb{F}_{2^{128}}$). We give syntax and security definitions for anonymous credit tokens, the construction — including a degree-16 Keccak checkpointing technique that commits the permutation state only every four rounds, and a batched quadratic-system constraint for whipped MQ verification — and a security analysis that states its assumptions (the ordinary MAYO OV and Multi-Target Whipped MQ problems; an ideal-XOF model) and its concrete attack ceilings explicitly. A Rust research prototype achieves 29–31 ms client proving per spend across the three proof-size profiles and 9.2–9.4 ms issuer verify-and-sign (proof verification plus MAYO preimage sampling) in the Balanced profile, with 71 KiB compact-profile proofs and 315-byte tokens.

1 Introduction

An anonymous credit token system has one issuer and many clients. The issuer grants a client a public number of credits; the client can later spend part of that balance without revealing which issuance — or which earlier spend — produced the token being presented. Each accepted spend consumes a one-time nullifier (preventing double spending) and returns a fresh token carrying the remaining balance. The functionality generalizes anonymous tokens in the Privacy Pass style [19] from unit tokens to stateful balances, and specializes keyed-verification anonymous credentials [18] to a single numeric attribute with exact arithmetic.

Classically this is well-trodden ground: blind signatures [16], CL signatures [17], algebraic MACs [18] and their pairing-based relatives give compact, efficient constructions. All of them lean on the algebraic structure of discrete-log or pairing groups, both for the signature and for

the efficient zero-knowledge presentation protocol. Neither survives a quantum adversary. The natural repair — a post-quantum *blind* signature — remains an active and unsettled research area in which several early lattice-based proposals required substantial revision after cryptanalysis; we prefer a design that does not need blindness at all.

The core idea: sign a digest, hide the preimage. VOLE-ACT, following the Katz–Schlesinger design [24], sidesteps blindness. A credential is a short XOF stream

$$X = \text{SHAKE256}(\text{domain} \parallel \text{ctx} \parallel \kappa \parallel \text{enc}_{64}(b) \parallel \rho),$$

over a client-chosen 256-bit nullifier key κ , public balance b , and 256-bit hiding nonce ρ . The first $4m$ bits of X form the *commitment digest* $C \in \mathbb{F}_{16}^m$; the next 256 bits form the *nullifier* N . At issuance the client *reveals* C and proves in zero knowledge that it is well-formed for the public balance b ; the issuer then chooses a fresh 256-bit salt ζ , computes $Y = \text{SHAKE256}(\text{"signed"} \parallel \text{pack16}(C) \parallel \text{enc}_{64}(0) \parallel \zeta)[0:4m]$, and returns a MAYO preimage $\sigma \leftarrow \text{SPre}(\text{sk}, Y)$ together with ζ . Nothing is blind: the issuer sees the commitment it authenticates. Unlinkability comes later — at spend time the client proves *in zero knowledge* that it knows $(\sigma, \kappa, b, \rho, \zeta)$ with $P^*(\sigma) = \text{SignedTarget}(C(\kappa, b, \rho), 0, \zeta)$, revealing only the nullifier N , the public spend amount, and a fresh commitment digest C' for the successor token. The signature and the signed digest are never shown again, so there is nothing to link. The unrevealed suffix of X is treated as pseudorandom given its prefix, which is what makes the nullifier unlinkable to the issuance in which the issuer saw C .

This pattern reduces the required primitives to (i) a post-quantum trapdoor map whose *verification* is cheap inside a proof system, and (ii) a post-quantum zero-knowledge proof system that is efficient on *hash-heavy Boolean* statements. The pairing of MAYO with VOLE-in-the-Head is unusually harmonious on both counts:

- **MAYO** [1, 3] verification evaluates a public multivariate quadratic map over $\mathbb{F}_{16}$. Since $\mathbb{F}_{16}$ embeds as a subfield of the proof system’s tag field $\mathbb{F}_{2^{128}}$, the whole verification equation becomes a single batched *quadratic system* constraint in the proof — no bit decomposition, no modular arithmetic emulation.
- **VOLE-in-the-Head** [4, 5] proof systems commit witness *bits* and prove Boolean and low-degree relations at QuickSilver [7, 8] cost: linear gates are free, and the entire batch of polynomial constraints of maximum degree d is checked with d published $\mathbb{F}_{2^{128}}$ elements. Keccak — the only hash used anywhere — is the friendliest possible circuit for such a system: its round function is degree 2.

Contributions. (1) Syntax and game-based security definitions for anonymous credit tokens with issuer-settled (*deferred-return*) spends (Section 3), together with an explicit account of why a signer-salted target restores the ordinary MAYO proof strategy and avoids an adaptive *one-more-preimage* oracle assumption. (2) A post-quantum candidate construction from the ordinary MAYO foundation plus an ideal XOF (Section 4): one XOF stream yielding both commitment and nullifier; a typed two-format token state machine; and a deferred-return settlement in which the issuer supplies a bounded return after the client fixes its proved request and before signer-salt generation, bound by the common hash. (3) Proof-circuit techniques (Section 5): *degree-16 Keccak checkpointing*, committing the permutation state only every four rounds and quartering the dominant witness cost by spending the proof system’s polynomial-degree budget; and a

batched quadratic-system constraint expressing whipped MQ verification over shared product terms, folded by verifier randomness. (4) A security analysis (Section 6) that is explicit about its conditional status, its concrete error terms, and its generic attack ceilings — including a $2^{256/3}$ -query quantum collision bound on the credential digest. (5) Evidence of practicality: a Rust research prototype (Section 7) with 29–31 ms spend proving across profiles, 9.2–9.4 ms Balanced-profile issuer verify-and-sign, 71 KiB compact proofs, and 315-byte tokens.

2 Preliminaries

2.1 Notation

$\lambda = 128$ throughout. $\mathbb{F}_2, \mathbb{F}_{16} = \mathbb{F}_2[x]/(x^4+x+1)$ and $\mathbb{F}_{2^{128}} = \mathbb{F}_2[x]/(x^{128}+x^7+x^2+x+1)$ are the Boolean, MAYO, and tag fields; $\iota : \mathbb{F}_{16} \hookrightarrow \mathbb{F}_{2^{128}}$ is the protocol-specified field embedding (the subfield is unique; the embedding is fixed deterministically, with the reduction polynomial family used by FAEST [5]). SHAKE256 is used for every PRG, commitment, and Fiat–Shamir hash, always with prefix-free domain separation and fixed-width or length-framed inputs [13]. $\text{enc}_{64}(\cdot)$ is 64-bit little-endian encoding. κ denotes the client’s nullifier key, h a GGM tree depth, and k_w MAYO’s whipping parameter.

2.2 VOLE-based zero knowledge and VOLE-in-the-Head

In VOLE-based ZK [8, 7], the prover holds, for each witness bit $u_t \in \{0, 1\}$, a *tag* $V_t \in \mathbb{F}_{2^{128}}$, and the verifier holds a global secret $\Delta \in \mathbb{F}_{2^{128}}$ and per-coordinate *keys*

$$K_t = V_t + u_t \cdot \Delta,$$

a vector oblivious linear evaluation (VOLE) correlation. Wires are affine combinations of committed bits and are linearly homomorphic for free. A multiplication (degree-2) constraint $a \cdot b = c$ is checked QuickSilver-style: the verifier forms $B = K_a K_b + \Delta K_c$, which the prover can match with a degree-1 polynomial in Δ exactly when the constraint holds; a random linear combination with challenge weights χ_i batches all constraints into one polynomial identity checked at Δ . The generalization to degree- d polynomial constraints costs d published coefficients and $d - 1$ random VOLE masks in total for the whole batch: mask $j \in \{0, \dots, d - 2\}$ is a tag-field VOLE pair contributing the degree-1 polynomial $(v_j + u_j \Delta) \Delta^j$, of degree at most $d - 1$ in total, so the leading, constraint-carrying coefficient is left untouched. Lemma 1 makes the masking exact: the published vector releases the check equation and nothing else.

Lemma 1 (Coefficient masking). *Fix Δ and the mask keys $K_j = v_j + u_j \Delta$ for $j \in \{0, \dots, d - 2\}$ (each mask group packed into one $\mathbb{F}_{2^{128}}$ element), and let the honest prover publish $c = (c_0, \dots, c_{d-1}) \in \mathbb{F}_{2^{128}}^d$, the coefficient vector of its masked response polynomial. Conditioned on the rest of the verifier’s view, c is uniform on the affine hyperplane $\{c : \sum_{i < d} c_i \Delta^i = A\}$, where A is the value the verifier computes for the check equation.*

Proof. The blinds added to the unmasked coefficients are $b_0 = v_0$, $b_j = v_j + u_j \Delta$ for $1 \leq j \leq d - 2$, and $b_{d-1} = u_{d-2}$. Substituting $v_j = K_j + u_j \Delta$ (characteristic 2) makes b an affine function of $(u_0, \dots, u_{d-2})$, injective by back-substitution: b_{d-1} determines u_{d-2} , then each b_j determines u_{j-1} from u_j , and b_0 is then fixed. Its evaluation is constant: $\sum_j b_j \Delta^j = \sum_j (v_j + u_j \Delta) \Delta^j = \sum_j K_j \Delta^j$, known to the verifier. So the image is exactly the hyperplane $\{b : \sum_j b_j \Delta^j = \sum_j K_j \Delta^j\}$ of

size $|\mathbb{F}_{2^{128}}|^{d-1}$, an injective affine image of $(u_0, \dots, u_{d-2}) \in \mathbb{F}_{2^{128}}^{d-1}$ — which stays uniform given the keys, since each pair (u_j, v_j) is uniform on the line $v_j = K_j + u_j \Delta$ — hence uniform on it; translating by the fixed unmasked coefficients preserves uniformity on the shifted hyperplane, whose defining value is the check value A . $\square$

VOLE-in-the-Head [4] makes this *publicly verifiable and non-interactive*: the prover generates the VOLE correlation itself from τ GGM [11] seed trees of depth h (with $\tau \cdot h = \lambda$), commits to all 2^h leaves of each tree with an all-but-one vector commitment, and the Fiat–Shamir challenge Δ selects, per tree, one leaf that stays hidden; the remaining leaves are opened and the verifier recomputes its keys. Soundness comes from the hidden leaf; zero knowledge from the fact that the opened leaves are exactly what the verifier can recompute anyway. Our instantiation follows the hardening of FAEST [5] and “One Tree to Rule Them All” [6]: every PRG and commitment call binds a per-proof 2λ -bit salt, a per-tree derived salt, and the exact tree position, preventing the known multi-target and multi-instance amortization attacks on the 128-bit seeds; small-field consistency across the τ trees uses public corrections re-based onto one shared u -vector, in the style of SoftSpokenOT [10].

Three parameter profiles trade proof size against tree-expansion work: *compact* $(\tau, h) = (16, 8)$, *balanced* $(32, 4)$, and *low-latency* $(64, 2)$. Each opened proof reveals τ tree openings of h sibling seeds, so proofs shrink as h grows while the prover expands $\tau \cdot 2^h$ leaves.

The consistency check must be a matrix hash. Because each tree contributes only h bits of Δ , a cheating prover who mangles one tree’s correction data is caught by a per-tree check only with probability $1 - 2^{-h}$ — as low as $3/4$ for $h = 2$. The construction therefore applies a 128-row *column-wise linear universal hash* to the entire tag matrix (two independent polynomial evaluations combined with random weights, plus a one-time-pad block), so that any inconsistency survives into the check equation except with probability at most $2^{-\lambda} + ((L-1)/2^\lambda)^2$ per column for L hashed chunks (Lemma 2; $L \approx 186$ for a direct spend, making the second term $\approx 2^{-241}$), regardless of the chunk structure. Composing this per-column bound with the challenge schedule and the Fiat–Shamir transform is part of the outstanding analysis. A scalar hash in place of the matrix hash is a real soundness bug we caught during development; we flag it because the failure is silent and parameter-dependent.

Lemma 2 (Wide-hash universality). *For keys $r_0, r_1, s, t \leftarrow \$\mathbb{F}_{2^{128}}$ define $H(x_0, x_1) = r_0 P_s(x_0) + r_1 P_t(x_0) + x_1$, where $x_0 \in \mathbb{F}_{2^{128}}^L$ is parsed as polynomial coefficients, $P_s(x_0) = \sum_i x_0[i] s^{L-1-i}$, and $x_1 \in \mathbb{F}_{2^{128}}$. For every $\delta_0 \neq 0$ and every $c \in \mathbb{F}_{2^{128}}$,*

$$\Pr[r_0 P_s(\delta_0) + r_1 P_t(\delta_0) = c] \leq 2^{-\lambda} + \left(\frac{L-1}{2^\lambda}\right)^2,$$

so H is ε -almost-universal in its x_0 argument with this ε ; and for uniform x_1 the output is uniform, so the pad block makes the published hash perfectly hiding.

Proof. $P_s(\delta_0)$ is the evaluation at s of a fixed nonzero polynomial of degree at most $L - 1$, so $\Pr_s[P_s(\delta_0) = 0] \leq (L-1)/2^\lambda$, and independently for t . Conditioned on $(A, B) = (P_s(\delta_0), P_t(\delta_0)) \neq (0, 0)$, the value $r_0 A + r_1 B$ is uniform over the choice of the weight multiplying a nonzero coordinate, so it hits c with probability $2^{-\lambda}$. Adding the probability of the event $(A, B) = (0, 0)$ — in which the left side is 0 and the bound must absorb $c = 0$ — gives the claim. $\square$

2.3 MAYO

MAYO [1, 3] is an oil-and-vinegar [12] trapdoor map: a public quadratic map $P : \mathbb{F}_{16}^n \rightarrow \mathbb{F}_{16}^m$ vanishing on a secret o -dimensional oil space, with $o < m$ deliberately too small for direct Kipnis–Shamir-style attacks; the “whipping” construction evaluates k_w correlated copies to stretch the oil space, so the effective (whipped) public map is $P^* : \mathbb{F}_{16}^{k_w n} \rightarrow \mathbb{F}_{16}^m$, matching the notation of the MAYO literature. The trapdoor samples preimages σ with $P^*(\sigma) = y$ for arbitrary targets y ; this *hash-then-invert* usage (rather than MAYO’s message-signing API) is exactly what the scheme needs. We use the round-2 parameter set MAYO₂: $(n, m, o, k_w) = (81, 64, 17, 4)$, so $\sigma \in \mathbb{F}_{16}^{324}$ (162 bytes) and targets are $m = 64$ nibbles (32 bytes). We note the risk posture explicitly: multivariate trapdoors have a difficult history — Rainbow fell to a practical attack in 2022 [2] — and MAYO’s whipping is comparatively young. The protocol also uses a one-hash, signer-salted wrapper rather than MAYO’s byte-for-byte message-signing API; Remark 4 records the resulting reduction boundary. Exact spend retries replay one durable response, while repeated authorized issuance samples a new salt and hence a new target except on a charged collision. The design compartmentalizes this risk: a MAYO break forges credentials but is not expected to deanonymize past transactions, whose privacy rests only on the hash-based layer (Claim 2).

For the proof circuit, the whipped map is expanded once per public key into m upper-triangular forms $G_a \in \mathbb{F}_{16}^{k_w n \times k_w n}$ with $P^*(\sigma)_a = \sigma^\top G_a \sigma$; the circuit then proves m quadratic equations over the *same* $\binom{k_w n + 1}{2}$ product terms (Section 5.2). The G_a table is derived locally and never transmitted: at $m \binom{k_w n + 1}{2}$ nibbles — about 1.7 MB for MAYO₂, sixteen times the wire-format public key of Table 1 — it is a real cost for constrained verifiers (open problem 2 in Section 9).

3 Definitions and assumptions

We state the syntax and the security games the eventual analysis must target; a fully formal treatment of concurrent sessions and adaptive corruptions is left to the reduction effort. Throughout the experiments: the application label **app** and the parameter, profile, and balance-width identifiers are fixed public constants; **KGen** is MAYO trapdoor generation; **CanonParse** is the scheme’s canonical public-key decoder, returning $\perp$ on any non-canonical encoding; and **DeriveCtx(pk)** is **Setup**’s context derivation applied to those constants and **pk**. The maps **C**, **SignedTarget**, **digest**, and **DeriveCtx** are all defined over one shared prefix-consistent ideal XOF $\mathcal{H}$ (Remark 5), which every experiment implicitly samples and provides to all parties, including the adversary.

Definition 1 (Anonymous credit token scheme). *An anonymous credit token scheme with balance space $\{0, \dots, 2^{64} - 1\}$ is a tuple of interactive algorithms*

$$(\text{Setup}, \text{Issue}, \text{Spend}, \text{SpendDeferred}, \text{VerifyToken})$$

between an issuer and clients:

- **Setup**($1^\lambda, \text{app}$) $\rightarrow (\text{sk}, \text{pk}, \text{ctx})$: *generates the issuer trapdoor, public key, and a context binding the key, application domain, parameter set, and protocol version.*
- **Issue**($\text{C}(\text{pk}, b), \text{I}(\text{sk}, b)$) $\rightarrow T$ (*public balance b*): *the client obtains a direct token T of effective balance b , or aborts.*
- **Spend**($\text{C}(\text{pk}, T, s), \text{I}(\text{sk})$) $\rightarrow T'$ (*public amount s*): *consumes T ’s nullifier and yields a direct token T' of effective balance $\text{bal}(T) - s$, or aborts.*

- $\text{SpendDeferred}(\langle C(\text{pk}, T, s), I(\text{sk}) \rangle) \rightarrow T'$: as Spend , except the issuer selects a return $t' \leq s$ (by its return policy, a session input) after the client fixes its proved request and before the signer salt; the output is a deferred-return token of effective balance $\text{bal}(T) - s + t'$.
- $\text{VerifyToken}(\text{pk}, T) \rightarrow \{0, 1\}$: local client authentication of a decoded token. Tokens are never sent as presentations; only spend transcripts are.

Both spend operations accept either token format as input; the four input/output transitions carry distinct type tags bound into their transcripts.

Definition 2 (Correctness, informal). Consider an experiment in which an environment drives an honest issuer and honest clients to build a DAG of tokens: roots arise from $\text{Issue}(b)$ and every internal node from spending a not-yet-spent node, with amounts respecting balances. The scheme is correct if, except with negligible probability, no honest operation aborts, every client-side authentication check passes, every output balance obeys the equations of Definition 1, distinct accepted inputs yield distinct nullifiers (colliding only with probability about $q^2/2^{257}$ over q accepted credentials, by the birthday bound on the 256-bit suffix), and exact retransmission of a spend request returns the stored response rather than a new node (issuance carries no retry cache: an accepted re-issuance resamples the preimage, but the opening and nullifier are unchanged, so no new lineage arises).

Definition 3 (Fiscal soundness). For a scheme Π and adversary $\mathcal{A}$, the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{fiscal}}(\lambda)$ of Figure 1 runs $\mathcal{A}$ as all clients against an honest issuer. Π is fiscally sound if for every efficient $\mathcal{A}$, $\text{Adv}_{\Pi, \mathcal{A}}^{\text{fiscal}}(\lambda) := \Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{fiscal}}(\lambda) = 1]$ is negligible.

$\text{Exp}_{\Pi, \mathcal{A}}^{\text{fiscal}}(\lambda)$
$(\text{sk}, \text{pk}, \text{ctx}) \leftarrow \text{Setup}(1^\lambda, \text{app})$ $I \leftarrow 0; \quad R \leftarrow 0; \quad \text{St} \leftarrow \emptyset; \quad \text{win} \leftarrow 0$ run $\mathcal{A}^{\text{olssue}, \text{oSpend}, \text{oSpendDef}}(\text{pk}, \text{ctx})$ return win
$\text{olssue}(b)$
run the issuer side of Issue at balance b with $\mathcal{A}$ as client if the issuer accepts: $I \leftarrow I + b$; return its response return $\perp$
$\text{oSpend}(req)$
if req does not parse as a typed ordinary-settlement request: return $\perp$ $(N, s) \leftarrow$ its nullifier and amount; $dig \leftarrow \text{digest}(\text{ctx}, req)$ if $\text{St}[N] = (dig', resp')$ exists: if $dig' = dig$: return $resp'$ // exact retry: no accounting change return $\perp$ // conflicting reuse of a consumed nullifier if issuer verification of req fails: return $\perp$ $resp \leftarrow$ issuer response (fresh preimage on the fresh target) $\text{St}[N] \leftarrow (dig, resp); \quad R \leftarrow R + s$ if $R > I$: $\text{win} \leftarrow 1$ return $resp$
$\text{oSpendDef}(req, t')$
if req does not parse as a typed deferred-settlement request: return $\perp$ $(N, s) \leftarrow$ its nullifier and amount; $dig \leftarrow \text{digest}(\text{ctx}, req)$ if $\text{St}[N] = (dig', resp')$ exists: if $dig' = dig$: return $resp'$ // exact retry replays the stored t' ; accounting unchanged return $\perp$ if issuer verification of req fails $\vee \neg(0 \leq t' \leq s)$: return $\perp$ $resp \leftarrow$ issuer response on the deferred target, recording t' $\text{St}[N] \leftarrow (dig, resp); \quad R \leftarrow R + s - t'$ if $R > I$: $\text{win} \leftarrow 1$ return $resp$

Figure 1: The fiscal-soundness experiment. Accounting is per logical spend event (first durable consumption of a nullifier); letting $\mathcal{A}$ choose $t' \leq s$ models issuer return policy conservatively. External redemption of value must be idempotent at the same store boundary; symmetrically, since I counts every independently authorized issuance call and issuance carries no retry cache, a deployment must replay ambiguous delivery under the same external idempotency key rather than re-sign or re-charge — issuance idempotency lives at the payment boundary.

Remark 1 (Redemption-only is without loss of generality). *The win condition counts only redeemed value, not outstanding spendable balances, so an attack that merely forks a token into two live successors has not yet won. It extends to a win at will: any adversary holding more spendable value than the invariant allows simply continues querying `oSpend` to cash the excess out, and the game runs for the whole experiment. Formally, redemption-only soundness is equivalent to the ledger invariant of the design document (redeemed value + live independently spendable value $\leq$ issued value) up to this cash-out reduction, which is the statement a full analysis should prove as a lemma rather than assume.*

Definition 4 (Spend anonymity, single challenge). *For a scheme Π and adversary $\mathcal{A}$ (a malicious issuer), the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{anon}}(\lambda)$ of Figure 2 is defined with the convention that the experiment's explicit abort steps (an invalid public key or an invalid challenge selection) output a uniformly random bit, so degenerate strategies gain nothing; a malformed final guess is likewise replaced by a fresh coin. Aborts the adversary induces inside protocol sessions are not experiment aborts. All handle state (spent, retired, exposed) initializes to 0, and `exposed[h]` is set as soon as a spend request on h is delivered to $\mathcal{A}$, whether or not the session completes: a spend request reveals the token's deterministic nullifier, so a candidate whose nullifier $\mathcal{A}$ has already seen would be trivially distinguishable. Π is spend-anonymous if for every efficient $\mathcal{A}$, $\text{Adv}_{\Pi, \mathcal{A}}^{\text{anon}}(\lambda) := |\Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{anon}}(\lambda) = 1] - \frac{1}{2}|$ is negligible.*

$\text{Exp}_{\Pi, \mathcal{A}}^{\text{anon}}(\lambda)$
$\beta \leftarrow \$ \{0, 1\}; \quad \mathcal{T} \leftarrow \emptyset \quad // \text{ handle table of honest tokens}$ $\text{pk}_{\text{bytes}} \leftarrow \$ \mathcal{A}(1^\lambda)$ $\text{pk} \leftarrow \text{CanonParse}(\text{pk}_{\text{bytes}}); \quad \text{if } \text{pk} = \perp : \text{abort}$ $\text{ctx} \leftarrow \text{DeriveCtx}(\text{pk}) \quad // \text{ as in Setup; fixed hereafter}$ $(h_0, h_1, s, \text{mode}) \leftarrow \$ \mathcal{A}^{\text{oClientOp}}()$ if $\neg \text{ValidChallenge}(h_0, h_1, s, \text{mode})$: abort $\text{retire}(h_0); \text{retire}(h_1) \quad // \text{ retired before the challenge session, so no oracle can touch them concurrently}$ run the honest client side of the mode-mode spend of amount s on $\mathcal{T}[h_\beta]$, with $\mathcal{A}$ as issuer; $\mathcal{A}$ sees the transcript, and may abort the session $// \text{ an issuer-induced challenge abort stays in the game}$ if the session accepts: $\text{spent}[h_\beta] \leftarrow 1$; store the successor at a fresh handle h^* ; $\text{retire}(h^*)$ $\beta' \leftarrow \$ \mathcal{A}^{\text{oClientOp}}(); \quad \text{if } \beta' \notin \{0, 1\} : \beta' \leftarrow \$ \{0, 1\}$ return $[\beta' = \beta]$
$\text{oClientOp}(\text{op}, h, \text{args})$
if $\text{op} \notin \{\text{Issue}, \text{Spend}, \text{SpendDeferred}\}$: return $\perp$ if $\text{op} \neq \text{Issue} \wedge (h \notin \mathcal{T} \vee \text{retired}[h] \vee \text{spent}[h])$: return $\perp$ run the honest client side of op on $\mathcal{T}[h]$ where required, with $\mathcal{A}$ as issuer, performing every client-side authentication check; if $\text{op} \neq \text{Issue}$: set $\text{exposed}[h] \leftarrow 1$ when the spend request is delivered to $\mathcal{A}$ if the session accepts: if $\text{op} \neq \text{Issue}$: $\text{spent}[h] \leftarrow 1$ store the output token at a fresh handle h' ; return h' return $\perp \quad // \text{ aborted sessions yield no token, but exposure persists}$
$\text{ValidChallenge}(h_0, h_1, s, \text{mode})$
return $\text{mode} \in \{\text{Spend}, \text{SpendDeferred}\} \wedge h_0 \neq h_1$ $\wedge \mathcal{T}[h_0], \mathcal{T}[h_1] \text{ exist, } \neg \text{retired}, \neg \text{exposed}, \text{authenticated} \quad // \text{ spent} \Rightarrow \text{exposed}$ $\wedge \text{same credential kind} \wedge \text{bal}(\mathcal{T}[h_0]) = \text{bal}(\mathcal{T}[h_1])$ $\wedge s \text{ valid for both} \quad // \text{ implied by equal balances; stated for clarity}$

Figure 2: The single-challenge spend-anonymity experiment against a malicious issuer. The explicit “abort” steps (invalid key, invalid challenge selection) output a uniformly random bit; an issuer-induced abort of the challenge *session* is ordinary protocol behavior — the transcript stands and $\mathcal{A}$ still guesses, so selective-failure strategies remain in scope. Handle-state conventions (initialization and the exposure trigger) are part of Definition 4; “authenticated” in **ValidChallenge** is implied by storage-on-accept and stated for emphasis.

Remark 2 (Design of the anonymity game). *The same-kind, same-mode, same-amount, equal-balance restrictions reflect leakage the syntax makes public by design. Retiring the candidates closes the immediate distinguisher: replaying a candidate would reveal β through nullifier reuse*

(and requiring unexposed candidates extends this to nullifiers revealed by aborted pre-challenge sessions). Retiring the successor isolates the definition to a single challenged transcript rather than smuggling in a compositional guarantee about continued use. This is scope hygiene, not a defense against a known distinguisher: with equal candidate balances the successor’s balance $\text{bal} - s + t'$ is a deterministic function of values $\mathcal{A}$ already holds — it chose t' itself, even if as a function of the observed nullifier — so we know of no attack through a live successor, and a proof that none exists is exactly the compositional statement the stronger definition below would supply. A chain definition (hidden, permuted token handles with identical admissible continuations) is the natural strengthening and is left open. Section 6 discusses the metadata any deployment leaks outside this game.

Assumption 1 (Ordinary MAYO foundations). *The OV problem and Multi-Target Whipped MQ problem of the MAYO Round-2 specification are hard at the selected parameters. This is what we mean below by “ordinary MAYO hardness”: OV hides the structured oil subspace, while Multi-Target Whipped MQ asks for a preimage of one of many random targets under a random whipped map. The latter is multi-target but gives the adversary no trapdoor-inversion oracle, and therefore is strictly different from the one-more game in Figure 3.*

$\text{Exp}_{\mathcal{A}}^{\text{omp}}(\lambda)$	
$(\text{sk}, \text{pk}) \leftarrow \text{\$ KGen}(1^\lambda); \quad \text{ctx} \leftarrow \text{DeriveCtx}(\text{pk}); \quad \mathcal{D} \leftarrow \emptyset$	
$\{(y_j, w_j, \sigma_j)\}_{j=1}^k \leftarrow \text{\$ } \mathcal{A}^{\mathcal{H}, \text{olnv}}(\text{pk})$	
return 1 iff $k \geq \mathcal{D} + 1 \wedge y_1, \dots, y_k$ pairwise distinct $\wedge \forall j : \text{Adm}(y_j, w_j) = 1 \wedge P^*(\sigma_j) = y_j$	
$\text{olnv}(y, w)$	$\text{Adm}(y, w)$
if $\text{Adm}(y, w) = 0$: return $\perp$ $\mathcal{D} \leftarrow \mathcal{D} \cup \{y\}$ return $\text{SPre}(\text{sk}, y)$ <i>// fresh randomness per call</i>	if $y \notin \mathbb{F}_{16}^m$: return 0 <i>// below: $\kappa, \rho \in \{0, 1\}^{256}, b, t \in \{0, \dots, 2^{64}-1\}$</i> if $w = (\kappa, b, \rho) \wedge y = \text{C}(\kappa, b, \rho)$: return 1 if $w = (\kappa, b, \rho, t)$ $\wedge y = \text{D}(\text{ctx}, \text{C}(\kappa, b, \rho), t)$: return 1 return 0

Figure 3: The adaptive one-more-preimage experiment required by the prior unsalted direct-target design, shown for contrast and *not* assumed by the construction below. The oracle power absent from ordinary MAYO is the ability to request trapdoor preimages of adversarially selected admissible targets.

Remark 3 (Why the one-more game is stronger). *The prior design passed a client-derived target directly to SPre. A simulator that chose a public preimage σ could evaluate $P^*(\sigma)$, but could not force that image to equal the already fixed client target; Figure 3’s inversion oracle supplied exactly the missing power. Merely proving that the client evaluated Keccak does not change this direction of simulation. PoMFRIT’s optimized construction uses an (n, q) one-more assumption for this reason. Its more general hash-and-prove construction instead relies on ordinary MAYO security.*

Remark 4 (Signer-salted wrapper and proof boundary). *The construction removes the problematic client-fixed-target interface and same-target multiplicity constructively: the issuer signs only salted wrappers*

$$y = \mathcal{H}(\text{“signed”} \parallel \text{pack16}(\mathsf{C}) \parallel \text{enc}_{64}(t) \parallel \zeta)[0 : 4m], \quad \zeta \leftarrow_{\$} \{0, 1\}^{256}$$

freshly after each request is accepted (with $t = 0$ for the direct format). The token stores ζ , and the spend relation proves the wrapper with ζ hidden. Except for salt prediction/collision, the XOF point is fresh, so the ordinary MAYO simulation can choose σ , compute $P^*(\sigma)$, and program that value at the salted point. An unsigned output can then feed the OV and Multi-Target Whipped MQ hops. Exact spend retries replay the durable (ζ, σ) pair and do not create another signing sample.

This is the ordinary MAYO proof shape, not a verbatim invocation of Round-2 Theorem 1: the wrapper encoding and salt derivation differ from Algorithm 7. An adapted classical-ROM proof must couple trapdoor sampling to public sampling, paying the published rank-rejection factor $(1 - Q_{\text{cpl}}B)^{-1}$ when $Q_{\text{cpl}}B < 1$, plus salt-prequery, collision, and local sampler-failure terms. For MAYO₂, $B \approx 2^{-20}$, so even the published proof has a per-key query boundary near 2^{19} signatures for a constant-factor loss. The 256-attempt implementation cap needs a distinct argument: Lemma 1 averages the rank-failure probability over key generation and a fresh vinegar sample, giving $\mathbb{E}[p_{\text{key}}] \leq B$, not a uniform per-key $p_{\text{key}} \leq B$. It therefore does not by itself justify a B^{256} cap-failure term for one reused key. A stronger tail or key-validation argument remains open. If Q_{try} is a hard experiment-level maximum on sampler invocations (each containing at most 256 rank attempts), the cited lemma alone gives only the conservative cap term $Q_{\text{try}}B$. The coupling uses $Q_{\text{cpl}} = Q_S$ only under response-oblivious race linearization and confidentiality of losing candidates. Merely replacing Q_S by Q_{try} does not simulate a winner selected through variable sampler timing; without the scheduling premise, a separate race-leakage lemma remains open. A QROM treatment also remains open. Omitting ctx from the outer encoding saves a rate block; C already commits it, and one MAYO key is confined to one derived context.

Remark 5 (Hash modeling and adversary class). *The analysis models SHAKE256 as an ideal XOF — a prefix-consistent variable-output random oracle, so that reading $4m+256$ bits and reading $4m$ bits of the same stream agree on their common prefix (ordinary length-indexed random-oracle outputs do not automatically provide this) — for three distinct roles: the credential stream (hiding, collision and preimage structure, suffix pseudorandomness given a prefix), the GGM/vector-commitment PRG layer, and Fiat–Shamir. FIPS 202 standardizes SHAKE but proves none of these properties; sponge indifferenciability is the standard heuristic bridge. The claims of Section 6 target classical PPT adversaries in this ROM; the primitives are post-quantum candidates, but a QROM treatment (quantum XOF access, quantum rewinding for extraction) is explicitly open, so “post-quantum” describes the assumption class, not a proved quantum security level.*

4 The VOLE-ACT construction

4.1 Credentials and nullifiers from one XOF stream

Fix a 256-bit context ctx binding the issuer key (by hash of the expanded public map), application domain, parameter set, VOLE profile, and protocol version. The client samples $\kappa, \rho \leftarrow \{0, 1\}^{256}$

and, for base balance $b \in \{0, \dots, 2^{64}-1\}$, derives the stream

$$X = \text{SHAKE256}(\text{"credential"} \parallel \text{ctx} \parallel \kappa \parallel \text{enc}_{64}(b) \parallel \rho), \quad C = X[0 : 4m], \quad N = X[4m : 4m+256].$$

For MAYO₂, C is $4m = 256$ bits; its width as a collision domain matters for fiscal security (Section 6). Both values come from a *single* sponge evaluation — the message fits one Keccak rate block — so proving both costs one in-circuit permutation. The commitment prefix C is revealed to the issuer exactly once (at signing time); the nullifier suffix N is revealed exactly once (at spending time). Under the ideal-XOF modeling, the suffix is pseudorandom given the prefix, which is what divorces the spend from the issuance.

4.2 One authenticated format, two typed settlement states

Every token is $(\sigma, \kappa, b, \rho, t, \zeta)$, where ζ is a uniform 256-bit signer salt and

$$\begin{aligned} P^*(\sigma) &= \text{SignedTarget}(C(\kappa, b, \rho), t, \zeta), \\ \text{SignedTarget}(C, t, \zeta) &= \text{SHAKE256}(\text{"signed"} \parallel \text{pack16}(C) \parallel \text{enc}_{64}(t) \parallel \zeta)[0 : 4m]. \end{aligned}$$

Its effective balance is $b + t$ (exact, non-wrapping). A *direct* token constrains $t = 0$; a *deferred-return* token carries an issuer-chosen t . (The formal tuple elides fixed metadata the implementation stores and `VerifyToken` checks: the derived context and local credential-kind marker.) At $t = 0$ the two markers intentionally name the same semantic credential: retagging that token does not create a new nullifier lineage or effective balance. The common hash gives a simple binding of the base commitment, return, and signer salt: changing t requires a fresh preimage or a target collision, and no algebraic combiner (such as $C + \iota(t)$) has to be analyzed for malleability at all.

Spends are typed. An ordinary `Spend` consumes either format and always outputs a direct token; `SpendDeferred` consumes either format and outputs a deferred-return token. The client commits to a fresh (κ', ρ', b') *before* the issuer chooses $t' \leq s$, which is precisely the case that cannot be expressed as a smaller net spend — if the return were known in advance, the client would simply spend less. Requests, proof statements, pending states, settlement responses, and retry records carry distinct mode tags, but the wire header is not itself an authenticator: retagged identical-layout bodies can parse. Proof statements and request digests supply end-to-end mode binding. Zero-return token and response aliases are deliberate and fiscally inert. A spend of amount $s = 0$ is deliberately admissible: it consumes the old nullifier and rotates (κ, ρ) , an unlinkable *refresh* of an unchanged balance — useful, for example, after suspected exposure of client state. (Each refresh writes a durable nullifier record at zero redeemed value, so deployments should rate-limit or price it.)

4.3 Proof relations

The construction proves membership in three NP relations. Define the helper maps (over the shared ideal XOF $\mathcal{H}$; `pack16` packs an $\mathbb{F}_{16}$ -vector two nibbles per byte, first element in the low nibble):

$\text{CredStream}_{\text{ctx}}(\kappa, b, \rho)$	$\text{Target}(T)$
$X \leftarrow \mathcal{H}(\text{"credential"} \parallel \text{ctx} \parallel \kappa \parallel \text{enc}_{64}(b) \parallel \rho)$ $C \leftarrow X[0:4m] \text{ as } \mathbb{F}_{16}^m$ $N \leftarrow X[4m:4m+256]$ return (C, N)	$(C, \cdot) \leftarrow \text{CredStream}_{\text{ctx}_T}(\kappa_T, b_T, \rho_T)$ return $\text{SignedTarget}(C, t_T, \zeta_T)$
$\text{SignedTarget}(C, t, \zeta)$	
return $\mathcal{H}(\text{"signed"} \parallel \text{pack16}(C) \parallel \text{enc}_{64}(t) \parallel \zeta)[0:4m]$	
$\text{VerifyToken}(\text{pk}, T)$	
if $\text{ctx}_T \neq \text{DeriveCtx}(\text{pk})$: return 0 if $\text{kind}(T) = \text{Direct} \wedge t_T \neq 0$: return 0 if $b_T + t_T \geq 2^{64}$: return 0 return $[P^*(\sigma_T) = \text{Target}(T)]$	

Figure 4: Helper maps and local token authentication. A token carries the tuple $(\text{kind}, \sigma, \kappa, b, \rho, t, \zeta)$, with $t = 0$ for the direct format, together with its stored context ctx_T ; the effective balance is $\text{bal}(T) = b_T + t_T$. The maps are written recomputing CredStream throughout; implementations cache the derived commitment, but a token *stores* none — C is always recomputed from the opening, so no stale commitment/opening mismatch can arise.

The issuance relation states that a revealed digest opens to the public balance; the two spend relations state possession of a preimage for the input token’s target, correct nullifier derivation, exact balance conservation, and well-formedness of the fresh commitment:

$$\begin{aligned}
\mathcal{R}_{\text{issue}} &= \{ (\text{ctx}, b, C); (\kappa, \rho) : (C, \cdot) = \text{CredStream}_{\text{ctx}}(\kappa, b, \rho) \} \\
\mathcal{R}_{\text{spend}}^{\text{Dir}} &= \{ (\text{ctx}, s, N, C'); (\sigma, \kappa, b, \rho, t, \zeta, e, \kappa', b', \rho') : \\
&\quad (C, N) = \text{CredStream}_{\text{ctx}}(\kappa, b, \rho) \wedge t = 0 \wedge P^*(\sigma) = \text{SignedTarget}(C, t, \zeta) \\
&\quad \wedge b + t = e \wedge b' + s = e \text{ exactly (no 64-bit wrap)} \\
&\quad \wedge (C', \cdot) = \text{CredStream}_{\text{ctx}}(\kappa', b', \rho') \\
&\quad \} \\
\mathcal{R}_{\text{spend}}^{\text{Def}} &= \{ (\text{ctx}, s, N, C'); (\sigma, \kappa, b, \rho, t, \zeta, e, \kappa', b', \rho') : \\
&\quad (C, N) = \text{CredStream}_{\text{ctx}}(\kappa, b, \rho) \wedge P^*(\sigma) = \text{SignedTarget}(C, t, \zeta) \\
&\quad \wedge b + t = e \wedge b' + s = e \text{ exactly (both sums below } 2^{64}) \\
&\quad \wedge (C', \cdot) = \text{CredStream}_{\text{ctx}}(\kappa', b', \rho') \\
&\quad \}
\end{aligned}$$

The balance equations are exact 64-bit unsigned additions with zero final carry, proven over committed carry bits: there is no modular wraparound to exploit, and overspending is exactly the statement “the final carry is 1,” which the relation excludes. All witness scalars range over their stated domains $(\kappa, \rho \in \{0, 1\}^{256}; b, b', t \in \{0, \dots, 2^{64}-1\})$.

4.4 Protocol algorithms

Figures 5 and 6 instantiate Definition 1 against three black-box interfaces: the trapdoor map (P^* , SPre , KGen), the shared XOF $\mathcal{H}$, and a non-interactive zero-knowledge argument of knowledge $\Pi = (\text{Prove}, \text{Verify})$ for the relations of Section 4.3, where the full typed statement — including ctx and the credential-kind and settlement-mode tags — is bound into the challenge derivation. Section 5 instantiates Π by executing each relation as a Boolean circuit under the VOLEitH proof system; an implementation of the protocol layer needs only these interfaces. $\text{digest}(\text{ctx}, \text{req}) = \mathcal{H}(\text{"request-digest"} \parallel \text{ctx} \parallel \text{frame}(\text{req}))[0:256]$ is the exact-retry key.

Setup ($1^\lambda, \text{app}$)	
$(\text{sk}, \text{pk}) \leftarrow \$ \text{KGen}(1^\lambda)$ // MAYO trapdoor generation; expand whipped forms $h_{\text{pk}} \leftarrow \mathcal{H}(\text{"mayo-pk"} \parallel \text{canonical encoding of the expanded } P^*)$ $\text{ctx} \leftarrow \mathcal{H}(\text{"context"} \parallel \text{frame}(\text{app}) \parallel h_{\text{pk}} \parallel \text{parameter/profile identifiers} \parallel \text{balance width})[0:256]$ return $(\text{sk}, \text{pk}, \text{ctx})$	
Issue.Client (pk, ctx, b)	Issue.Issuer ($\text{sk}, \text{ctx}, b, (\text{C}, \pi)$)
$\kappa, \rho \leftarrow \$ \{0, 1\}^{256}$ $(\text{C}, -) \leftarrow \text{CredStream}_{\text{ctx}}(\kappa, b, \rho)$ $\pi \leftarrow \$ \Pi.\text{Prove}(\mathcal{R}_{\text{issue}}, (\text{ctx}, b, \text{C}), (\kappa, \rho))$ return $(\text{req} = (\text{C}, \pi), \text{st} = (\kappa, b, \rho, \text{C}))$	// b : externally authorized // session input, not in the request require $\Pi.\text{Verify}(\mathcal{R}_{\text{issue}}, (\text{ctx}, b, \text{C}), \pi)$ $\zeta \leftarrow \$ \{0, 1\}^{256}; \quad Y \leftarrow \text{SignedTarget}(\text{C}, 0, \zeta); \quad \sigma \leftarrow \$ \text{SPre}(\text{sk}, Y)$ return (ζ, σ)
Issue.Finish ($\text{pk}, \text{st}, (\zeta, \sigma)$)	
require $P^*(\sigma) = \text{SignedTarget}(\text{C}, 0, \zeta)$ return $T = (\text{Direct}, \sigma, \kappa, b, \rho, t=0, \zeta)$	

Figure 5: Setup and issuance.

Spend.Client (pk, ctx, T, s, mode)
require VerifyToken (pk, T) $\wedge$ $s \leq \text{bal}(T)$ $\kappa', \rho' \leftarrow_{\$} \{0, 1\}^{256}$; $b' \leftarrow \text{bal}(T) - s$ $(C', -) \leftarrow \text{CredStream}_{\text{ctx}}(\kappa', b', \rho')$; $(-, N) \leftarrow \text{CredStream}_{\text{ctx}}(\kappa_T, b_T, \rho_T)$ $x \leftarrow (\text{ctx}, \text{kind}(T), \text{mode}, s, N, C')$ $e_T \leftarrow b_T + t_T$; $w \leftarrow (\sigma_T, \kappa_T, b_T, \rho_T, t_T, \zeta_T, e_T, \kappa', b', \rho')$ $\pi \leftarrow_{\$} \text{II.Prove}(\mathcal{R}_{\text{spend}}^{\text{kind}(T)}, x, w)$ return ($\text{req} = (\text{kind}(T), \text{mode}, s, N, C', \pi)$, $st = (\kappa', b', \rho', C', s, \text{mode})$)
Spend.Issuer (sk, ctx, St, req, policy)
$dig \leftarrow \text{digest}(\text{ctx}, \text{req})$ if $\text{St}[\text{req}.N] = (dig', \text{resp}')$ exists : return resp' if $dig' = dig$ else $\perp$ $x \leftarrow (\text{ctx}, \text{req}.kind, \text{req}.mode, \text{req}.s, \text{req}.N, \text{req}.C')$ if $\text{req}.mode = \text{Spend}$: $t' \leftarrow 0$ else : $t' \leftarrow \text{policy}(\text{req})$ with $0 \leq t' \leq \text{req}.s$ require II.Verify ($\mathcal{R}_{\text{spend}}^{\text{req}.kind}, x, \text{req}.\pi$) $\zeta' \leftarrow_{\$} \{0, 1\}^{256}$; $Y' \leftarrow \text{SignedTarget}(\text{req}.C', t', \zeta')$; $\sigma' \leftarrow_{\$} \text{SPre}(\text{sk}, Y')$ $\text{resp} \leftarrow (t', \zeta', \sigma')$ <i>atomically</i> insert $\text{St}[\text{req}.N] \leftarrow (dig, \text{resp})$; let (dig^*, resp^*) be the durable record return resp^* if $dig^* = dig$ else $\perp$
Spend.Finish (pk, st, resp)
$(t', \zeta', \sigma') \leftarrow \text{resp}$; require $P^*(\sigma') = \text{SignedTarget}(C', t', \zeta')$ if $st.mode = \text{Spend}$: require $t' = 0$; return (Direct , $\sigma', \kappa', b', \rho', 0, \zeta'$) require $t' \leq st.s$; return (Deferred , $\sigma', \kappa', b', \rho', t', \zeta'$)

Figure 6: Spending, both settlement modes. `policy` is the issuer’s return-selection function; the durable-record comparison implements exact-retry replay and conflicting-reuse rejection under concurrent insertion.

The client-side requirement `VerifyToken` and the finish-time checks are the complete authentication surface the anonymity game relies on. An ordinary spend of a deferred-return token folds the old return into b' and emits the common format with $t' = 0$; the issuer verifies, signs, and durably stores the response under N , so exact retransmissions replay and a network failure cannot burn a balance.

5 The proof circuit

The relation above is almost entirely SHAKE256: every spend proves three hidden sponge evaluations (old stream, signed wrapper, fresh stream). Everything else — the MAYO equation and the balance arithmetic — is quadratic. This section describes the three techniques that make the circuit small.

5.1 Degree-16 Keccak checkpointing

Keccak- $f[1600]$'s round function is degree 2 (the χ step is the only nonlinearity) [13]. The naive circuit commits all 1600 state bits after every round: 24 rounds $\times$ 1600 witness bits, with degree-2 constraints between layers. But witness bits are the dominant cost in a VOLE-style proof (each consumes a VOLE coordinate, enlarging both the correction data and every leaf-PRG expansion), while raising the *degree* of the batched QuickSilver check is cheap: a maximum degree d costs d published field elements and $d - 1$ mask VOLEs *in total*.

We therefore commit the state only every *four* rounds. Between checkpoints the circuit carries symbolic polynomial expressions: after rounds 1, 2, 3, 4 the expressions have degree 2, 4, 8, 16, and the checkpoint constraint asserts (per bit) that the degree-16 expression equals the next committed state — a degree-16 polynomial constraint, the largest the instantiation accepts. This quarters the per-permutation witness from 24×1600 to $6 \times 1600 = 9600$ bits. One permutation dominates each hidden SHAKE evaluation, so both spend witnesses are 31,760 bits for MAYO₂ — versus roughly 118,200 under per-round checkpointing. The cost is the coefficient-vector products of the expression layer, about 1.14 million $\mathbb{F}_{2^{128}}$ multiplications per permutation — and the *verifier* pays the same evaluation over its keys: measured on the prototype, this expression layer is the dominant term of issuer verification, ahead of the quadratic-system folds and the seed-tree reconstruction (Section 7).

5.2 The quadratic-system constraint for MAYO

Verifying $P^*(\sigma) = y$ means checking $m = 64$ quadratic equations

$$\sum_{r \leq c} G_a[r, c] \sigma_r \sigma_c = y_a, \quad a \in [m],$$

over the *same* $\binom{325}{2} = 52,650$ products $\sigma_r \sigma_c$. Materializing each product as a committed wire would multiply the witness; instead the proof system supports a *deferred quadratic system*: the prover records the values and tags of every term factor, and at challenge time the m equations are folded into a single QuickSilver degree-2 check by random weights $\varphi_a \in \mathbb{F}_{2^{128}}$ drawn from the transcript, with coefficients lifted through the embedding ι . An unsatisfied equation survives the fold except with probability $2^{-\lambda}$. The committed signature nibbles enter as bit wires ($4k_w n = 1296$ bits). The same mechanism folds the 64 quadratic carry equations of each balance addition.

5.3 Balance arithmetic in $\mathbb{F}_2$

Additions are proven bitwise with committed carry vectors: $a_i + b_i + c_i = s_i$ linearly (over $\mathbb{F}_2$, addition is XOR — an early design error we caught was writing balance equations additively over the tag field, where they say nothing about integers), and the carry recurrence $c_{i+1} = a_i b_i + c_i(a_i + b_i)$ quadratically via the folded system. Asserting the final carry to be zero makes every balance equation an exact statement about 64-bit unsigned integers, so conservation is enforced bit-exactly rather than modulo anything.

6 Security analysis

We state the intended results as conditional claims against the definitions of Section 3; no complete reduction is claimed, and where the implementation's own analysis states sharper

caveats, those govern.

Claim 1 (Fiscal soundness, conditional). *In the ideal-XOF model of Remark 5, under Assumption 1 and the signer-salted wrapper reduction sketched in Remark 4, adaptive multi-theorem straight-line knowledge soundness (online extractability from the random-oracle transcript, without rewinding, at proof acceptance and before current-output signing) of the VOLEitH argument for both the issuance and the spend relations, a linearizable, non-rollback nullifier store with confidential race losers and response-oblivious winner selection, and service-level atomic issuance authorization, charging, idempotent durable recording, and first publication, VOLE-ACT satisfies Definition 3.*

Discussion and error terms. Conditionally on extraction, every accepted operation yields an opening and a MAYO preimage for an admissible target, correct nullifier derivation, and exact balance arithmetic. Nullifier-class accounting turns over-redemption, outside collision and store failures, into an EUF-CMA forgery for the common salted wrapper: an authenticated message descriptor (C, t) not previously submitted to the wrapper signer. The reduction halts on the first such extracted input before signing the current output, so no current or later query can retroactively submit that message. Remark 4 sketches how the ordinary MAYO simulation handles this forgery without a one-more oracle. Two credential-stream openings with equal C are a bad event in the fiscal-to-unsigned-message lift. A collision between an unsigned wrapper input and a programmed signing target need not be a separate abort in the later MTWMQ game: the transferred preimage itself solves that random target. It still exposes the generic ceiling of the same $4m$ -bit output space, and a finished advantage statement must place the event explicitly. A further operational collision event: the retry store retains a 256-bit *request digest* rather than the full request, so exact-retry semantics additionally assume collision resistance of that digest (or full-request comparison), and the redeemed service itself — not merely the token response — must be tied idempotently to the first durable record. Nullifier collisions, by contrast, need no fiscal accounting: two distinct accepted credentials with colliding N only make the store refuse whichever spends second — a liveness failure charged to correctness (Definition 2), never extra value. Issuance soundness matters because it ties every signed wrapper message to an extracted opening; without it, a malicious issuance request could obtain a signature on an unaccounted descriptor. The dominant known terms and generic ceilings include (this is not yet a complete advantage expression — proof-attempt counts, Q_H factors, vector-commitment and consistency terms, and composition loss are unquantified):

- The degree-16 batched QuickSilver check contributes up to $17/2^{128} \approx 2^{-123.9}$ per proof attempt, before vector-commitment, consistency-hash, Fiat–Shamir, and composition terms.
- The credential digest is a collision domain of only $4m = 256$ bits. Two openings $(\kappa, b, \rho) \neq (\kappa', b', \rho')$ with the same prefix C let the *one* issuer-supplied preimage σ be presented under both — their suffix nullifiers will generally differ — double-spending, and with distinct balances creating value. This is generic collision search: about 2^{128} classically, and query complexity $\Theta(2^{256/3}) \approx 2^{85.3}$ for an ideal quantum collision adversary (upper bound from BHT [14], under its demanding memory model; matching query lower bound from Aaronson–Shi [15]). Deployments requiring a larger quantum collision margin need a wider digest, hence a larger m . Any concrete birthday term must count all distinct credential-XOF inputs, including adversarial offline grinding as well as honest and extracted openings; it cannot count only the two inputs appearing in the final collision.

Honest double spends are prevented operationally: a repeated credential reveals the same deterministic nullifier, whose conflicting reuse the store refuses while exact retransmissions are replayed from the stored record — the atomic retry semantics that also make honest retransmission safe.

Claim 2 (Spend anonymity, conditional). *In the ideal-XOF model of Remark 5, assuming the VOLEitH argument is adaptive multi-theorem non-interactive zero knowledge for all three chosen-key relations $\mathcal{R}_{\text{issue}}$, $\mathcal{R}_{\text{spend}}^{\text{Dir}}$, and $\mathcal{R}_{\text{spend}}^{\text{Def}}$ (i.e., against maliciously formed but canonically parsed public keys, in the ROM), VOLE-ACT satisfies Definition 4; the claim does not depend on any MAYO assumption.*

Discussion. A spend reveals (s, N, C', proof) . The intended argument: the compiled argument is NIZK zero-knowledge for all three relations in the sense above (the 2λ -bit per-proof salt is multi-instance *hardening*, not by itself a theorem upgrading honest-verifier ZK to this property) — zero knowledge of the *issuance* proof is as load-bearing as that of the spend proofs, since it argues about the very stream whose unrevealed suffix is the nullifier, and a leaky issuance proof would link the later spend with no MAYO involvement; C' hides its opening under the fresh 256-bit nonce ρ' ; and N is the unrevealed suffix of an XOF stream whose prefix the issuer saw at issuance, pseudorandom given the prefix in the ideal-XOF model. None of these steps is currently backed by a proof for this exact composition, so the claim is conditional on all three. Its MAYO-independence is the design’s compartmentalization: a trapdoor recovery or forgery does not help the distinguisher in Definition 4. Outside the game, deployments leak metadata the definition deliberately excludes: spend amounts are public; direct-input and deferred-input proofs now have the same circuit and byte size, but the explicit credential-kind and settlement tags, timing, network identity, and retry behavior remain visible. Two further channels deserve explicit flags. The definition hands every session one pk , so key *consistency* is assumed, not provided: a malicious issuer serving different (individually canonical) keys to different clients partitions the anonymity set completely, since each spend verifies under exactly one derived ctx — deployments need key transparency or another consistency mechanism, as in Privacy-Pass-style systems. And unlike the passive channels above, the deferred return is *active*: an issuer can watermark a client by choosing a distinctive t' , which is exactly what the equal-balance restriction on challenge candidates excludes from the game — return policies should be deterministic public functions of public data (say, of s and the credential kind).

Multi-instance hardening. Every PRG and commitment call in the tree layer binds the per-proof salt, the per-tree derived salt, and the exact tree position (after [6]; Section 2.2), preventing the known multi-target and multi-instance amortization attacks on the 128-bit seeds; a concrete multi-user bound still carries adversary-instance-dependent terms. The wide consistency hash removes the 2^{-h} per-chunk cheating strategy. Domain tags separate every hash use. Encodings are canonical within each requested typed decoder, which rejects alternate encodings of that typed value. Identical-layout bodies can be header-retagged into another local marker, however: wire tags are discriminants rather than authenticators, while proof statements, request digests, and semantic checks provide end-to-end binding.

7 Parameters and evaluation

The construction is implemented as a Rust research prototype (five crates: binary fields, GGM vector commitments, the VOLEitH proof system, MAYO, and the protocol). Table 1 gives wire

sizes; tokens are profile-independent and only proof-carrying requests grow with the low-latency profiles. On one Apple M4 Pro, the common spend relation proves in 29–31 ms across profiles. Criterion measured Balanced-profile issuer verify-and-sign at 9.2–9.4 ms for spends (issuance: 3.37 ms); no cross-profile issuer range is inferred from that run. Direct and deferred inputs now differ only by zero constraints and have identical proof sizes. Figures are statistically sampled central estimates on an idle machine. The compact profile’s proofs are $3.8\times$ smaller than low-latency. Implementation techniques, optimization methodology, and full per-profile measurements are documented in the repository, <https://github.com/SamuelSchlesinger/vole-act> (docs/IMPLEMENTATION.md, docs/BENCHMARKS.md; the latter includes the 2026-07-21 signer-salted experiment and a labeled pre-wrapper baseline).

Table 1: Wire sizes (bytes), MAYO₂, $\lambda = 128$. Spend responses are uniformly framed — the direct mode carries the deferred mode’s return field structurally zero. Responses and tokens additionally carry a 32-byte signer salt. The public-key row is the transmitted expanded MAYO map; the locally derived G_a table of Section 2.3 (≈ 1.7 MB) is never sent.

Object	compact (τ, h)=(16, 8)	balanced (32, 4)	low-latency (64, 2)
Token (either format)	315	315	315
Issue request	29 750	55 286	106 358
Spend request (either input)	73 086	141 918	279 582
Issue / spend response	203 / 211	203 / 211	203 / 211
Public key (expanded map)	106 319	106 319	106 319

8 Related work

VOLE-based designated-verifier ZK begins with Wolverine [8] and Mac’n’Cheese [9]; Quick-Silver [7] gives the degree- d batched check we generalize. VOLE-in-the-Head [4] removes the designated verifier and underlies the FAEST signature candidate [5], whose engineering (GGM trees, salting, consistency checking) we follow, with the hardening of [6]. MAYO [1, 3] descends from unbalanced oil-and-vinegar [12]; [2] is the cautionary tale for the assumption class. The closest post-quantum relative is PoMFRIT [23], a *blind-signature* scheme that likewise combines VOLE-in-the-Head proofs of SHA-3-family circuits with the MAYO trapdoor. Its general construction proves the hash/signature relation and uses ordinary MAYO security; its optimized direct-target construction introduces the one-more-preimage assumption. This distinction motivates our signer-salted wrapper. PoMFRIT pursues the blindness route with the same toolbox, while VOLE-ACT’s commit-then-sign design avoids blindness and adds stateful balances, typed settlement, and deferred returns. Two other post-quantum routes give contrast. Generic succinct proof systems yield credentials by proving possession of a conventionally signed credential (the zk-creds template [20]); instantiated with hash-based, STARK-style proofs the route is plausibly post-quantum and far more succinct than ours, but with a substantially heavier prover on exactly the hash statements VOLEitH proves natively, and with no analogue of the free $\mathbb{F}_{16} \subseteq \mathbb{F}_{2^{128}}$ treatment of the trapdoor equation. Structured lattices give practical anonymous credentials and round-optimal blind signatures [21, 22] with compact transcripts under algebraic lattice assumptions; against both routes, VOLE-ACT trades succinctness for a symmetric-heavy assump-

tion base, a fast transparent prover, and exact balance arithmetic, at the price of 71 KiB proofs and an online stateful issuer. Anonymous tokens and credentials span blind signatures [16], CL signatures [17], keyed-verification credentials from algebraic MACs [18], and deployed unit-token systems [19]; VOLE-ACT is a post-quantum candidate stateful-balance member of this family, instantiating the Katz–Schlesinger design [24].

9 Open problems

(1) A complete security reduction: formalizing Definitions 3–4 fully (concurrency, chains, adaptive corruptions) and proving Claims 1 and 2 in the QROM, including the exact salted-wrapper adaptation of MAYO’s OV/MTWMQ proof and its rejection-sampling loss. (2) A structured, non-materialized QuickSilver check for the whipped MAYO equation, removing the $O(m(k_w n)^2)$ expanded-form table (≈ 1.7 MB for MAYO₂; Section 2.3) and enabling the larger MAYO parameter sets (whose wider digests also raise the quantum collision ceiling). (3) Hashing the compact public-key encoding (rather than the expanded forms) into the context — a transcript-breaking cleanup best done before any format freeze. (4) Aggregating or compressing spend proofs; at 71 KiB the compact profile is deployable server-side but remains the dominant bandwidth cost. (5) Hidden spend amounts, which would require in-circuit range checks and re-open the balance-width trade-off.

References

- [1] W. Beullens. *MAYO: Practical Post-Quantum Signatures from Oil-and-Vinegar Maps*. SAC 2021.
- [2] W. Beullens. *Breaking Rainbow Takes a Weekend on a Laptop*. CRYPTO 2022.
- [3] W. Beullens, F. Campos, S. Celi, B. Hess, M. J. Kannwischer. *MAYO: Algorithm Specification (Round-2 document, February 5, 2025)*. NIST PQC Additional Digital Signatures. This work pins the Round-2 specification; MAYO advanced to Round 3 in May 2026. <https://pqmayo.org/resources/>.
- [4] C. Baum, L. Braun, C. Delpech de Saint Guilhem, M. Klooß, E. Orsini, L. Roy, P. Scholl. *Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head*. CRYPTO 2023.
- [5] C. Baum, W. Beullens, L. Braun, C. Delpech de Saint Guilhem, M. Klooß, C. Majenz, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, P. Scholl. *FAEST: Algorithm Specification, Version 2.0 (Round-2 document)*. NIST PQC Additional Digital Signatures. This work follows the Round-2 (v2.0) design; FAEST advanced to Round 3 in May 2026. <https://faest.info>.
- [6] C. Baum, W. Beullens, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, P. Scholl. *One Tree to Rule Them All: Optimizing GGM Trees and OWFs for Post-Quantum Signatures*. ASIACRYPT 2024.
- [7] K. Yang, P. Sarkar, C. Weng, X. Wang. *QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field*. ACM CCS 2021.
- [8] C. Weng, K. Yang, J. Katz, X. Wang. *Wolverine: Fast, Scalable, and Communication-Efficient Zero-Knowledge Proofs for Boolean and Arithmetic Circuits*. IEEE S&P 2021.
- [9] C. Baum, A. J. Malozemoff, M. B. Rosen, P. Scholl. *Mac’n’Cheese: Zero-Knowledge Proofs for Boolean and Arithmetic Circuits with Nested Disjunctions*. CRYPTO 2021.

- [10] L. Roy. *SoftSpokenOT: Quieter OT Extension from Small-Field Silent VOLE in the Minicrypt Model*. CRYPTO 2022.
- [11] O. Goldreich, S. Goldwasser, S. Micali. *How to Construct Random Functions*. Journal of the ACM 33(4), 1986.
- [12] A. Kipnis, J. Patarin, L. Goubin. *Unbalanced Oil and Vinegar Signature Schemes*. EUROCRYPT 1999.
- [13] NIST. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. FIPS PUB 202, 2015.
- [14] G. Brassard, P. Høyer, A. Tapp. *Quantum Cryptanalysis of Hash and Claw-Free Functions*. LATIN 1998. arXiv:quant-ph/9705002.
- [15] S. Aaronson, Y. Shi. *Quantum Lower Bounds for the Collision and the Element Distinctness Problems*. Journal of the ACM 51(4), 2004.
- [16] D. Chaum. *Blind Signatures for Untraceable Payments*. CRYPTO 1982.
- [17] J. Camenisch, A. Lysyanskaya. *An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation*. EUROCRYPT 2001.
- [18] M. Chase, S. Meiklejohn, G. Zaverucha. *Algebraic MACs and Keyed-Verification Anonymous Credentials*. ACM CCS 2014.
- [19] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, F. Valsorda. *Privacy Pass: Bypassing Internet Challenges Anonymously*. Proceedings on Privacy Enhancing Technologies 2018(3), 164–180.
- [20] M. Rosenberg, J. White, C. Garman, I. Miers. *zk-creds: Flexible Anonymous Credentials from zkSNARKs and Existing Identity Infrastructure*. IEEE S&P 2023.
- [21] J. Bootle, V. Lyubashevsky, N. K. Nguyen, A. Sorniotti. *A Framework for Practical Anonymous Credentials from Lattices*. CRYPTO 2023.
- [22] W. Beullens, V. Lyubashevsky, N. K. Nguyen, G. Seiler. *Lattice-Based Blind Signatures: Short, Efficient, and Round-Optimal*. ACM CCS 2023.
- [23] C. Baum, M. Beckmann, W. Beullens, S. Mukherjee, C. Rechberger. *Concretely Efficient Blind Signatures Based on VOLE-in-the-Head Proofs and the MAYO Trapdoor (PoMFRIT)*. USENIX Security 2026. IACR ePrint 2026/109.
- [24] J. Katz, S. Schlesinger. *Anonymous Credit Tokens*. IETF CFRG Internet-Draft draft-schlesinger-cfrg-act (work in progress), 2025; design document and Rust implementation, <https://github.com/SamuelSchlesinger/anonymous-credit-tokens> (accessed July 2026).